

Steps for Performing a Hypothesis Test

1. State the Hypotheses

- **Null Hypothesis(H_0):** The default assumption or claim to be tested (e.g., "There is no difference in means").
- **Alternative Hypothesis(H_1):** The claim you want to test for (e.g., "There is a difference in means").

2. Choose the Significance Level (α)

- Common choices are 0.05, 0.01, or 0.10.
- This is the threshold for deciding whether a result is statistically significant.

3. Select the Appropriate Test

- Decide which statistical test suits your data and hypothesis (e.g., t-test, z-test, chi-square test, ANOVA).

4. Calculate the Test Statistic

- Use the sample data to compute the test statistic (e.g., t, z, F, or chi-square value) based on the chosen test.

5. Compute the p-value

- The p-value is the probability of observing a test statistic as extreme as, or more extreme than, the one calculated (assuming the null hypothesis is true).

6. Compare the p-value to the Significance Level

- **If p-value $\leq \alpha$:** Reject the null hypothesis(H_0). There is sufficient evidence to support the alternative hypothesis.
- **If p-value $> \alpha$:** Fail to reject the null hypothesis(H_0). There is insufficient evidence to support the alternative hypothesis.

7. State the Conclusion

- Clearly state the result in the context of the problem, referencing whether the null hypothesis was rejected or not.

The Significance Level

Before running any test, you must decide: "How much risk of being wrong am I comfortable with?" That threshold is the **significance level (α)**. It is set before collecting data.

Think of α as a confidence dial:

- $\alpha = 0.05$ → "I'll accept a 5% chance I'm making a wrong call"
- $\alpha = 0.01$ → "I need to be 99% sure before I act"
- $\alpha = 0.10$ → "I'm okay being wrong 10% of the time as it's a low-stakes decision"

The significance level directly controls the probability of a Type I Error which is the most critical tradeoff in hypothesis testing.

Imagine you're a doctor testing a new medicine. You want to know if it really works better than the old one. To decide, you run an experiment with patients and collect data.

The Courtroom Analogy: Think of your experiment like a courtroom trial:

- Null Hypothesis (H_0): The new medicine is no better than the old one (the defendant is innocent).
- Alternative Hypothesis (H_1): The new medicine is better (the defendant is guilty).

In court, you don't want to wrongly convict an innocent person. In science, you don't want to wrongly claim your medicine works when it doesn't. This mistake is called a **Type I error**.

The Two Ways You Can Be Wrong

In hypothesis testing, you are making a binary decision (Reject H_0 or Fail to Reject H_0) about a truth you can never fully observe. This means there are exactly two types of mistakes you can make.

Think of it like a courtroom verdict:

	H_0 is Actually TRUE (Accused is Innocent)	H_0 is Actually FALSE (Accused is Guilty)
Reject H_0 (Verdict: Guilty)	Type I Error — Innocent person jailed	Correct Decision

H_0 is Actually TRUE (Accused is Innocent)	H_0 is Actually FALSE (Accused is Guilty)
Fail to Reject H_0 (Verdict: Not Guilty)	Type II Error — Guilty person freed
Correct Decision	

Type I Error : The False Alarm

A Type I Error occurs when you reject H_0 even though it is actually true, you see an effect that doesn't really exist. It is also called a False Positive.

- **Probability of Type I Error** = α (your chosen significance level)
- If $\alpha = 0.05$, you will make this mistake 5 in every 100 tests purely by chance

Type II Error : The Missed Signal

A Type II Error occurs when you fail to reject H_0 even though it is actually false — you miss a real effect that genuinely exists. It is also called a False Negative.

- **Probability of Type II Error** = β (beta)
- **Statistical Power** = $1 - \beta \rightarrow$ the probability of correctly detecting a real effect

p-value

The p-value is the probability of observing a test statistic as extreme as (or more extreme than) the one you got, assuming H_0 is true.

A small p-value means: "If H_0 were true, seeing this result would be very unlikely — so maybe H_0 isn't true."

✓ Z-Test

A z-test is a statistical method used in hypothesis testing to determine if there is a significant difference between sample and population means, or between the means of two samples. It is particularly useful when the population standard deviation is known and the sample size is large (typically greater than 30).

We have mainly two types of z-test:

1. **One-Sample Z-Test:** Used to determine whether the mean of a single sample is different from a known population mean.
2. **Two-Sample Z-Test:** Used to compare the means of two independent samples to see if they are significantly different from each other.

Assumptions of Z-Tests

For a z-test to yield valid/correct results, these assumptions must be met:

1. **Normal Distribution:** The data should be approximately normally distributed. This assumption is satisfied with large sample sizes due to the central limit theorem.
2. **Known Population Standard Deviation:** The standard deviation of the population must be known. If it is unknown, using a **t-test** is more appropriate.
3. **Random Sampling:** The sample data should be randomly drawn from the population, ensuring that it is representative of the actual population data.
4. **Independence:** The samples must be independent of each other, particularly in two-sample z-tests.
5. **Continuous Data:** The z-test is applicable for continuous data, where the variable of interest can take any numeric value.

✓ 1. One-Sample Z-test (One sample test for known pop variance)

Scenario 1: Manufacturing Quality Control

A pharmaceutical company produces a specific tablet that must contain exactly 500 mg of an active ingredient. Historical data from their high-precision machines indicates a population standard deviation (σ) of 10 mg. A Quality Assurance (QA) engineer randomly samples 50 batches and finds the average content is 496 mg. They need to determine if the machine has drifted out of calibration.

Null Hypothesis (H_0): The machine is calibrated correctly. The mean content is equal to the target.

$$H_0 : \mu = 500$$

Alternative Hypothesis (H_1): The machine is out of calibration. The mean content is not equal to the target.

$$H_1 : \mu \neq 500$$

```
import numpy as np
from scipy import stats

# Given Data
population_mean = 500
population_std = 10
sample_size = 50
sample_mean = 496

# Calculate Z-Score
# Formula: (sample_mean - pop_mean) / (pop_std / sqrt(n))
z_statistic = (sample_mean - population_mean) / (population_std / np.sqrt(sample_size))

# Calculate P-Value (Two-tailed test)
p_value = stats.norm.sf(abs(z_statistic)) * 2

print(f"Z-Score: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

if p_value < 0.05:
    print("Reject Null Hypothesis: The machine has drifted out of calibration.")
else:
    print("Fail to Reject Null: The deviation is likely due to chance.")
```

```
Z-Score: -2.8284
P-Value: 0.0047
Reject Null Hypothesis: The machine has drifted out of calibration.
```

Scenario: Quality Assurance (Food Manufacturing)

Problem Statement: A large food processing plant produces "Energy Bars" that are advertised to weigh exactly 50 grams. The strict quality control policy states that the production line standard deviation is known to be 2 grams due to mechanical variances.

A Quality Assurance Data Scientist randomly samples 100 bars from today's production batch and finds the average weight is 49.5 grams. They need to test if the machinery has drifted significantly from the target weight of 50g.

Hypothesis Statements:

- **Null Hypothesis (H_0):** The average weight of the bars is equal to the target. (The machine is working correctly).

$$H_0 : \mu = 50$$

- **Alternative Hypothesis (H_1):** The average weight of the bars is not equal to the target. (The machine is miscalibrated).

$$H_1 : \mu \neq 50$$

```

import numpy as np
import scipy.stats as stats

# 1. Setup Data
pop_mean = 50
pop_std_dev = 2
sample_size = 100
sample_mean = 49.5

# Simulate the sample data (Mean is shifted to 49.5 to simulate a problem)
sample_data = np.array([50.40483822, 46.8761864 , 48.88114199, 51.30141821, 49.33128064,
    46.09371437, 50.14253783, 52.62251794, 51.48639237, 51.76837878,
    48.68765281, 50.161313 , 48.32608142, 49.34135682, 47.9729816 ,
    49.28430948, 51.73274552, 50.25960079, 48.7359658 , 47.13827422,
    49.71008232, 52.70339994, 46.66162822, 53.3433419 , 52.78276636,
    53.43381614, 49.63156066, 46.22869873, 46.73314012, 48.98528066,
    47.50090802, 49.75194656, 51.97821175, 51.29384476, 48.98192443,
    50.72538558, 48.00899826, 52.47304215, 49.33947076, 49.04525544,
    49.22226128, 48.01008311, 49.25012366, 46.82159139, 49.99530902,
    47.71889532, 49.68001116, 48.09053674, 48.82212249, 46.63723591,
    49.14591185, 47.97374877, 46.49607605, 50.50171503, 48.0091587 ,
    51.18485864, 48.94155265, 54.12698446, 50.75096089, 54.31920821,
    46.54053288, 48.53904457, 49.61707527, 48.66808338, 50.55790802,
    50.10122835, 47.79250497, 50.45203966, 51.7998047 , 47.50426235,
    47.84189047, 49.02526641, 48.2429361 , 49.19689951, 51.54477141,
    49.88651678, 50.97091732, 49.42268857, 49.63775632, 49.81267013,
    49.54469218, 49.17206591, 49.52264168, 52.81964232, 49.18501505,
    51.04107237, 49.6800387 , 51.60434694, 49.8592259 , 49.21821781,
    50.13179064, 49.44307184, 42.19376068, 52.79645355, 50.35153978,
    48.09518794, 50.83571755, 47.71391235, 50.35067636, 53.36916694])

# 2. Perform One-Sample Z-Test
# 'value' is the mean under the Null Hypothesis
#z_stat, p_value = ztest(sample_data, value=population_mean_target)

z_stat = (sample_mean - pop_mean) / (pop_std_dev / (np.sqrt(sample_size)))

p_val = stats.norm.cdf(z_stat)

# 3. Interpret Results
alpha = 0.05
print(f"Z-Statistic: {z_stat:.4f}")
print(f"P-Value: {p_val:.4f}")

if p_val < alpha:
    print("Reject Null Hypothesis: The machinery is miscalibrated (Significant difference).")
else:
    print("Fail to Reject Null Hypothesis: The production is within acceptable limits.")

Z-Statistic: -2.5000
P-Value: 0.0062
Reject Null Hypothesis: The machinery is miscalibrated (Significant difference).
```

Note:

- When we have $\neq$ in Alternate Hypothesis then we reject the null hypothesis if $p_value * 2 < 5\%$.
- And if we have $<$ or $>$ symbols in alternate hypothesis then we reject the null hypothesis if $p_value < 5\%$.

Scenario : Resting Heart Rate from Raw Data (Left-Tailed)

Statement: Resting heart rate in a healthy adult population averages 72 bpm ($\sigma = 9$ bpm, known from a large population study). A researcher records resting heart rate for 30 patients on a new wellness program and wants to know if their average differs from the population.

Hypothesis:

- **Null Hypothesis(H_0):** $\mu = 72$
- **Alternative Hypothesis(H_1):** $\mu < 72$
- **Significance Level(α)** = 0.05

```
import numpy as np
import scipy.stats as stats

heart_rates = [70, 75, 68, 74, 71, 69, 73, 77, 66, 72,
               75, 70, 68, 74, 71, 76, 69, 73, 72, 70,
               74, 68, 71, 75, 69, 73, 70, 72, 76, 67]

sample_mean = np.mean(heart_rates)
n = len(heart_rates)

pop_mean = 72
pop_std = 9
alpha = 0.05

z_stat = (sample_mean - pop_mean) / (pop_std / np.sqrt(n))

p_value = stats.norm.cdf(z_stat)

decision = "Reject H0" if p_value < alpha else "Fail to reject H0"

print(f"Z-statistic : {z_stat:.4f}")
print(f"Sample Mean : {sample_mean}")
print(f"P-value      : {p_value:.4f}")
print(f"Decision     : {decision} (alpha = {alpha})")
```

```
Z-statistic : -0.2434
Sample Mean : 71.6
P-value      : 0.4038
Decision     : Fail to reject H0 (alpha = 0.05)
```

Problem Statement: Microsoft Azure's SLA guarantees a mean monthly uptime of 99.9% for its virtual machines. An enterprise client's IT team suspects the actual uptime has deviated from this guarantee. Based on Azure's historical infrastructure data, the population standard deviation is $\sigma = 0.15\%$. A sample of 60 monthly uptime records is collected across different regions.

Null Hypothesis (H_0): $\mu = 99.9\% \rightarrow$ Mean uptime equals the Azure SLA guarantee.

Alternative Hypothesis (H_1): $\mu \neq 99.9\% \rightarrow$ Mean uptime differs significantly from the SLA.

```

import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

pop_mean = 99.9          # SLA guaranteed uptime (%)
pop_std = 0.15           # Known population SD (%)
n = 60                   # Sample size
alpha = 0.05             # Significance level

# Simulated uptime records (slightly below SLA)
sample = np.array([ 99.73318097, 99.54485253, 99.93008179, 99.88838492,
                    99.62562335, 99.94755451, 99.86221944, 99.78921275,
                    99.86881594, 99.73168465, 99.84886035, 99.78691716,
                    99.89210638, 99.93330534, 99.6333859 , 99.87250222,
                    99.64243078, 99.69738179, 99.85538024, 99.90125539,
                    99.50327471, 99.96225267, 100.          , 99.93485648,
                    99.95015347, 99.73300337, 99.77957957, 100.          ,
                    99.64075896, 100.          , 99.66100324, 99.62387793,
                    99.6022992 , 99.85202612, 99.72272869, 99.85600779,
                    99.84120076, 99.78748201, 99.80899409, 99.8956776 ,
                    99.84084225, 100.          , 99.72547921, 99.86130348,
                    100.          , 100.          , 99.78222582, 99.71688945,
                    99.71300042, 99.6078916 , 99.68238411, 99.77273031,
                    99.6688727 , 99.91532552, 99.93267535, 99.94189772,
                    99.78791748, 99.93393658, 100.          , 99.79510054])

sample_mean = sample.mean()
print(f"Sample mean ( $\bar{x}$ ):{sample_mean:.5f}%")
print()

#Calculate Z-statistic
SE = pop_std / np.sqrt(n)
z_stat = (sample_mean - pop_mean) / SE

print(f"Standard Error(SE) = {SE:.2f}")
print(f"Z = (sample mean - population mean) / SE")
print(f"Z = ({sample_mean:.2f} - {pop_mean}) / {SE:.2f}")
print(f"Z = {z_stat:.4f}")
print()

# Calculate p_value
p_one_tail = stats.norm.cdf(z_stat)
p_val = 2 * p_one_tail
z_critical = stats.norm.ppf(1 - alpha / 2)

print()
print(f"Z-statistic : {z_stat:.4f}")
print(f"Z-critical :  $\pm$ {z_critical:.4f}")
print(f"p-value : {p_val:.4f}")
print()

if p_val < alpha:
    print("Reject  $H_0$  ( $p < 0.05$  AND Z-statistic beyond Z_critical)")
    print(f"Uptime ({sample_mean:.4f}%) is significantly different from the 99.9% SLA.")
else:
    print("Fail to Reject  $H_0$  ( $p \geq 0.05$ )")
    print("Uptime does not significantly deviate from the SLA guarantee.")

```

Sample mean ($\bar{x}$):99.81365%

Standard Error(SE) = 0.02
 $Z = (\text{sample mean} - \text{population mean}) / SE$
 $Z = (99.81 - 99.9) / 0.02$
 $Z = -4.4593$

Z-statistic : -4.4593
Z-critical : ± 1.9600
p-value : 0.0000

Reject H_0 ($p < 0.05$ AND Z-statistic beyond Z_critical)
Uptime (99.8136%) is significantly different from the 99.9% SLA.

Two Sample Z-test

Scenario 1: E-Commerce A/B Testing (Average Order Value)

A large online fashion retailer, "StyleHub," is testing a new checkout page design (Design B) against their current design (Design A). They believe the new design streamlines the process and encourages users to add more small accessories before paying. They have collected data from 200 randomly selected users for each design. The Data Science team needs to determine if there is a statistically significant difference in the Average Order Value (AOV) between the two designs.

Hypothesis Statements:

Let μ_A be the mean AOV of Design A and μ_B be the mean AOV of Design B.

- **Null Hypothesis (H_0):** There is no difference in the Average Order Value between the two designs.

$$H_0 : \mu_A - \mu_B = 0$$

- **Alternative Hypothesis (H_1):** There is a significant difference in the Average Order Value between the two designs.

$$H_1 : \mu_A - \mu_B \neq 0$$

```
import numpy as np
from statsmodels.stats.weightstats import ztest

# 1. Simulate Data (Large sample size > 30 implies Z-test is applicable)
np.random.seed(42)

# Design A: Mean $120, Std Dev $15, N=200
design_a_aov = np.random.normal(loc=120, scale=15, size=200)

# Design B: Mean $125, Std Dev $18, N=200
design_b_aov = np.random.normal(loc=125, scale=18, size=200)

# 2. Perform Two-Sample Z-Test
# value=0 implies we are testing for a difference of 0
z_stat, p_value = ztest(design_a_aov, design_b_aov, value=0)

# 3. Interpret Results
alpha = 0.05
print(f"Z-Statistic: {z_stat:.4f}")
print(f"P-Value: {p_value:.4f}")

if p_value < alpha:
    print("Reject Null Hypothesis: There is a significant difference in AOV between designs.")
else:
    print("Fail to Reject Null Hypothesis: No significant difference found.")
```

T-test

1. One-Sample t-test (One Sample test for unknown pop variance)

Problem : Zomato's Delivery Time Claim

Scenario: Zomato claims on their website that the average delivery time is 30 minutes. A data analyst randomly collects 25 delivery times from a city branch and suspects deliveries are actually taking longer. Is Zomato's claim statistically valid?

Hypotheses:

- **H_0 (Null):** $\mu = 30$ minutes (the mean delivery time equals the claimed 30 minutes)
- **H_1 (Alternate):** $\mu > 30$ minutes (the mean delivery time is greater than 30 minutes) → Right-tailed test

```
import numpy as np
from scipy import stats
from scipy.stats import ttest_1samp, shapiro

# Simulated sample: 25 delivery times (in minutes)
delivery_times = np.array([
    32, 35, 29, 38, 31, 34, 33, 40, 28, 36,
```

```

37, 30, 39, 33, 35, 31, 42, 34, 36, 38,
29, 33, 37, 35, 34])

t_statistic, p_value = ttest_1samp(delivery_times, popmean = 30)
print('p_value for 1-sample t-test', p_value)

```

```

import numpy as np
#from scipy import stats
from scipy.stats import ttest_1samp, shapiro

# Simulated sample: 25 delivery times (in minutes)
delivery_times = np.array([
    32, 35, 29, 38, 31, 34, 33, 40, 28, 36,
    37, 30, 39, 33, 35, 31, 42, 34, 36, 38,
    29, 33, 37, 35, 34])

#shapiro-wilk test for checking the normality of a sample
shapiro_stat, shapiro_p_value = shapiro(delivery_times)
print('Shapiro p_value is:', shapiro_p_value)

# run 1-sample t-test
t_statistic, p_value = ttest_1samp(delivery_times, popmean = 30, alternative = 'two-sided')
print('p_value for 1-sample t-test', p_value)

Shapiro p_value is: 0.953195335138537
p_value for 1-sample t-test 2.8829497629638653e-06

```

Note: If the sample data is not normally distributed, then we can run **Wilcoxon-Signed Rank test** as an alternative of 1 sample t-test.

Business Problem Statement

A beverage company claims that the average content of its soda cans is 330 ml. The quality control team wants to verify this claim by measuring the content of a random sample of cans.

Hypotheses

- **Null hypothesis (H_0):** The mean content in soda cans is 330 ml ($\mu = 330$).
- **Alternative hypothesis (H_1):** The mean content is not 330 ml ($\mu \neq 330$).

```

import numpy as np
from scipy.stats import ttest_1samp
from scipy.stats import shapiro

#sample data of 15 cans
sample_data = np.array([332, 329, 331, 328, 334, 330, 327, 333, 329, 331, 332, 328, 330, 334, 329])

shapiro_stat, p_value_shapiro = shapiro(sample_data)
print(p_value_shapiro)

#perform the t-test using ttest-1samp function
t_statistic, p_value = ttest_1samp(sample_data, popmean = 330, alternative = 'two-sided')

print(p_value)

```

Scenario: Call Center Efficiency (One-Sample T-Test)

Domain: Operations / Customer Service

A Call Center Operations Manager has set a Key Performance Indicator (KPI) for the "Average Handle Time" (AHT) of calls to be 300 seconds (5 minutes).

They implement a new training program for a small pilot group of 20 agents. After a week, the average AHT for this group is 285 seconds with a sample standard deviation of 35 seconds. The manager needs to know if this reduction is statistically significant or just random noise before rolling it out to the whole floor.

Hypothesis Statements

Null Hypothesis (H_0): The training had no effect. The mean AHT is still 300 seconds.

$$H_0 : \mu = 300$$

Alternative Hypothesis (H_1): The mean AHT is different from 300 seconds (The training changed performance).

$$H_1 : \mu \neq 300$$


```
import numpy as np
from scipy import stats

# Generating 20 data points with mean ~285 and std ~35
sample_data = np.array([319.7597332, 308.07830736, 334.9966668, 251.01243326,
                        321.41963416, 261.45445144, 234.85574711, 291.37489106,
                        275.95774222, 225.61724601, 304.78739298, 287.34189357,
                        245.76961858, 273.62817729, 317.77929274, 285.3406861,
                        250.91796348, 324.23487543, 311.57965774, 287.99525514])

# Perform One-Sample T-Test
t_stat, p_val = stats.ttest_1samp(sample_data, popmean = 300)

print(f"Sample Mean: {np.mean(sample_data):.2f}")
print(f"T-Statistic: {t_stat:.4f}")
print(f"P-Value: {p_val:.4f}")

# 3. Decision
alpha = 0.05
if p_val < alpha:
    print("Reject Null: The training significantly changed the handle time.")
else:
    print("Fail to Reject Null: The difference is likely due to chance.")
```

```
Sample Mean: 285.70
T-Statistic: -1.9725
P-Value: 0.0633
Fail to Reject Null: The difference is likely due to chance.
```

2. Two Sample t-test

Business Problem Statement

An HR department wants to compare the average monthly salaries of employees in two different departments (A and B) to determine if there is a significant difference.

Hypotheses

- Null hypothesis (H_0): The mean salary in Department A equals that in Department B ($\mu_1 = \mu_2$).
- Alternative hypothesis (H_1): The mean salaries are different ($\mu_1 \neq \mu_2$).

Before performing a two-sample t-test, it is important to check two key assumptions:

- Normality: Each sample should be approximately normally distributed.
- Equality of Variances: The two samples should have similar variances (for the standard t-test).

```
import numpy as np
from scipy.stats import shapiro, ttest_ind, levene, mannwhitneyu

# Monthly salaries in USD for two departments
dept_a = np.array([4500, 4700, 4200, 4800, 4600, 4400, 4550, 4650, 4750, 4300])
dept_b = np.array([4000, 4150, 4100, 3950, 4200, 4050, 4100, 4000, 4150, 4050])

# shapiro-wilk test on both the samples
shap_stat_a, shap_p_val_a = shapiro(dept_a)
shap_stat_b, shap_p_val_b = shapiro(dept_b)
print(shap_p_val_a, shap_p_val_b)

# levene test for equality of variances
levene_stat, p_value_levene = levene(dept_a, dept_b)
print('Levene test p_value: ', p_value_levene)

# perform independent 2-sample t-test
t_stat, p_value = ttest_ind(dept_a, dept_b, equal_var = False)
print(p_value)
```

```
0.7713665744080458 0.8485975564649946
Levene test p_value: 0.029281204462727414
1.4988271248674483e-05
```

1. The sample data

```
import numpy as np

# Monthly salaries in USD for two departments
dept_a = np.array([4500, 4700, 4200, 4800, 4600, 4400, 4550, 4650, 4750, 4300])
dept_b = np.array([4000, 4150, 4100, 3950, 4200, 4050, 4100, 4000, 4150, 4050])
```

2. Use the Shapiro-Wilk test to check if each sample follow a normal distribution.

```
from scipy.stats import shapiro

# Shapiro-Wilk test for normality
stat_a, p_a = shapiro(dept_a)
stat_b, p_b = shapiro(dept_b)

print(f"Dept A: p-value = {p_a:.4f}")
print(f"Dept B: p-value = {p_b:.4f}")

if p_a > 0.05:
    print("Dept A: Sample looks normal.")
else:
    print("Dept A: Sample does NOT look normal.")

if p_b > 0.05:
    print("Dept B: Sample looks normal.")
else:
    print("Dept B: Sample does NOT look normal.")
```

If the p-value of shapiro wilk test is greater than 0.05, the sample can be considered normally distributed.

3. Use Levene's test to check if the variances of the two samples are equal.

```
from scipy.stats import levene

# Levene's test for equal variances
stat_levene, p_levene = levene(dept_a, dept_b)

print(f"Levene's test p-value = {p_levene:.4f}")

if p_levene > 0.05:
    print("Variances are equal.")
    equal_var = True
else:
    print("Variances are NOT equal.")
    equal_var = False
```

If the p-value of levene test is greater than 0.05, you can assume equal variances

4. Choose the appropriate t-test based on the result of Levene's test.

```
from scipy.stats import ttest_ind

# Two-sample t-test (Welch's if variances unequal)
t_stat, p_value = ttest_ind(dept_a, dept_b, equal_var=False)

print(f"T-statistic: {t_stat:.2f}")
print(f"P-value: {p_value:.4f}")
```

If the p-value is less than 0.05, you can conclude there is a significant difference in means.

Problem Statement: A mobile developer study wants to assess whether apps on Apple's App Store have a significantly different average download size (in MB) compared to equivalent apps on Google's Play Store. A matched sample of 35 popular apps is measured on both platforms.

Null Hypothesis (H_0): $\mu_{\text{Appstore}} = \mu_{\text{Playstore}} \rightarrow$ There is no significant difference in mean app download size between the App Store and Play Store.

Alternative Hypothesis (H_1): $\mu_{\text{Appstore}} \neq \mu_{\text{Playstore}} \rightarrow$ There is a significant difference in mean app download size.

```
import numpy as np
from scipy.stats import shapiro, ttest_ind, levene, mannwhitneyu

# Size of 35 apps on apple app store
app_store = np.array([182.43269963, 179.87308557, 249.15559856, 238.1240509 ,
171.70416069, 171.97822097, 244.47863967, 228.46613734,
221.43478225, 143.95057309, 180.02670229, 143.53961326,
185.78465236, 158.76789235, 191.33576215, 211.29023925,
229.48626932, 234.6045203 , 220.90882774, 192.85590097,
215.71274633, 214.91522413, 186.74126932, 168.24893219,
191.48666105, 175.44279373, 247.48501844, 198.65883039,
119.0214585 , 174.85097299, 246.4411557 , 213.42397063,
238.76551845, 147.3449588 , 162.34451975])

play_store = np.array([159.33932351, 145.92827183, 188.65348808, 155.60218003,
162.52276931, 131.72707397, 140.15515561, 179.78346145,
147.12576383, 175.41494493, 209.66618593, 151.58965988,
136.94005162, 131.10585859, 131.30474009, 132.09321705,
174.35245445, 83.16266041, 170.35653133, 198.47742861,
186.68198936, 191.83370086, 182.0149168 , 67.47605909,
104.20073442, 153.45548316, 157.58809032, 184.48663111,
145.66015099, 155.83742675, 107.08213748, 163.56435429,
165.2219134 , 204.87391717, 144.9408706])

# shapiro-wilk test on both the samples
shap_stat_a, shap_p_val_a = shapiro(app_store)
shap_stat_b, shap_p_val_b = shapiro(play_store)
print(shap_p_val_a, shap_p_val_b)

# levene test for equality of variances
levene_stat, p_value_levene = levene(app_store, play_store)
print('Levene test p_value: ', p_value_levene)

# perform independent 2-sample t-test
t_stat, p_value = ttest_ind(app_store, play_store, equal_var = False)
print(p_value)

0.2787585701623423 0.2442351137179017
Levene test p_value: 0.3494386727314748
1.687944652520929e-06
```

Problem : Do Male & Female Customers Spend Differently?

Scenario: An e-commerce company wants to know if the average order value differs between male and female customers to decide if they should run gender-targeted promotions.

Hypotheses:

- **H_0 (Null):** $\mu_{\text{male}} = \mu_{\text{female}}$ (average spend is equal)
- **H_1 (Alternate):** $\mu_{\text{male}} \neq \mu_{\text{female}}$ (average spend differs) $\rightarrow$ Two-tailed test

```
import numpy as np
from scipy import stats

np.random.seed(99)

# Average order values (₹) for 35 customers each
male_spend = np.round(np.random.normal(loc=1450, scale=300, size=35), 2)
female_spend = np.round(np.random.normal(loc=1600, scale=280, size=35), 2)
```

Scenario 3: R&D Battery Testing (Two-Sample Independent T-Test), Domain: Manufacturing / Hardware Engineering

A tech company is selecting a battery supplier for their new smartwatch. Supplier A claims their battery lasts 24 hours. Supplier B is cheaper but claims similar performance. **Constraint:** Testing is expensive and time-consuming. The engineering team can only test 12 batteries from each supplier. Since $n < 30$, a Z-test is inappropriate; we must use a T-test.

Null Hypothesis (H_0): The average battery life of Supplier A and Supplier B is identical.

$$H_0 : \mu_A = \mu_B$$

Alternative Hypothesis (H_1): The average battery life differs between the two suppliers.

$$H_1 : \mu_A \neq \mu_B$$

```

from scipy import stats
import pandas as pd

# 1. Data (Hours of battery life)
# Small sample sizes (n=12)
supplier_a = [24.1, 23.8, 24.5, 23.0, 24.2, 23.9, 24.0, 23.5, 24.8, 24.1, 23.7, 24.3]
supplier_b = [22.8, 23.1, 22.5, 23.0, 22.9, 23.5, 22.1, 23.3, 22.8, 23.0, 22.7, 23.4]

# 2. Perform T-Test (Assuming equal variance for this hardware spec)
t_stat, p_val = stats.ttest_ind(supplier_a, supplier_b, equal_var=True)

print(f"Supplier A Mean: {sum(supplier_a)/len(supplier_a):.2f} hrs")
print(f"Supplier B Mean: {sum(supplier_b)/len(supplier_b):.2f} hrs")
print(f"P-Value: {p_val:.6f}") # Using 6f because p-value might be very small

# 3. Decision
if p_val < 0.05:
    print("Reject Null: Supplier A performs significantly better (or different).")
    # Interpretation: If A > B and significant, don't buy B even if it's cheaper!
else:
    print("Fail to Reject Null: No significant difference found.")

```

▼ ANOVA

Before performing a one-way ANOVA, it is essential to check the following key assumptions:

- Normality: Each group should be approximately normally distributed.
- Homogeneity of Variances: All groups should have similar variances.
- Independence: Observations should be independent (usually ensured by study design).

Scenario 1: Marketing Campaign Effectiveness Problem

Statement: A digital marketing team wants to determine which social media platform yields the highest user engagement score (0-100) for a new product launch. They run simultaneous campaigns on TikTok, Instagram, and LinkedIn and collect engagement scores from random users on each platform.

Hypothesis Formulation:

- **Null Hypothesis (H_0):** There is no significant difference in the mean engagement scores across the three platforms.

$$H_0 : \mu_{TikTok} = \mu_{Instagram} = \mu_{LinkedIn}$$

- **Alternative Hypothesis (H_1):** At least one platform has a mean engagement score different from the others.

```

import numpy as np
from scipy.stats import f_oneway, shapiro, levene, kruskal

tiktok = np.array([65.70876234, 75.24148111, 53.36094 , 88.31124168, 66.9498208 ,
62.7274252 , 84.95705692, 73.6630608 , 74.53086367, 80.41976844,
61.79220387, 74.1663686 , 84.6109146 , 65.48623431, 73.93197654,
94.78663015, 78.69687923, 63.32408083, 63.86821659, 64.15995633,
79.18828803, 66.04985321, 62.63865761, 77.99304857, 83.73391146,
69.32029347, 55.91996341, 51.52365563, 75.99622561, 69.89560614])

instagram = np.array([76.38405464, 83.48538268, 68.92500799, 77.15189002, 60.37999486,
54.9064827 , 84.64504956, 76.26070475, 51.35595813, 81.21324046,
75.16737898, 66.89979229, 76.945872 , 85.60134782, 58.56430539,
72.17314063, 52.95971012, 73.63052986, 63.54290254, 72.16740738,
79.10188442, 83.40293232, 70.20805162, 58.08966378, 80.21496265,
70.56094365, 70.68869961, 72.30945941, 76.05238316, 71.82903503])

linkedin = np.array([74.09639092, 62.94630478, 65.10572241, 81.18655794, 63.9444842 ,
74.01858283, 65.93010531, 78.05168414, 57.61019555, 65.94576657,
73.24615889, 85.56356553, 63.36566193, 65.25667837, 62.22340595,
76.34393328, 77.79381458, 57.40172342, 61.51665284, 63.05031077,
58.25281691, 66.58920115, 73.26048231, 40.97154251, 52.80879973,
67.46343802, 60.08994255, 60.4971181 , 59.24497195, 80.8282937 ])

```

```

print("--- Assumption Checks ---")

# 2. Check Assumption 1: Normality (Shapiro-Wilk Test)
# H0: The data is normally distributed.
# H1: The data is NOT normally distributed.
# We fail to reject H0 if p-value > 0.05

groups = {'TikTok': tiktok, 'Instagram': instagram, 'LinkedIn': linkedin}
normality_passed = True

for name, data in groups.items():
    stat, p = shapiro(data)
    print(f"{name} shapiro p-value: {p:.4f}")
    if p < 0.05:
        normality_passed = False
        print(f"--> Warning: {name} data may not be normally distributed.")

if normality_passed:
    print("Result: Normality assumption holds (all p > 0.05).")
else:
    print("Result: Normality assumption violated.")

print("\n")

```

```

--- Assumption Checks ---
TikTok shapiro p-value: 0.8886
Instagram shapiro p-value: 0.0931
LinkedIn shapiro p-value: 0.3852
Result: Normality assumption holds (all p > 0.05).

```

```

# 3. Check Assumption 2: Homogeneity of Variance (Levene's Test)

```

```

stat, p_levene = levene(tiktok, instagram, linkedin)
print(f"Levene's Test p-value: {p_levene:.4f}")

variance_passed = True
if p_levene < 0.05:
    variance_passed = False
    print("Result: Homogeneity of variance assumption violated.")
else:
    print("Result: Homogeneity of variance assumption holds.")

```

```

Levene's Test p-value: 0.6090
Result: Homogeneity of variance assumption holds.

```

```

# 4. Run ANOVA (Only if assumptions are reasonably met)
# Note: ANOVA is robust to slight deviations, but if assumptions fail badly,
# we should use Kruskal-Wallis (Non-parametric).

```

```

if normality_passed and variance_passed:
    print("Assumptions met. Running One-Way ANOVA...")

    f_stat, p_value = f_oneway(tiktok, instagram, linkedin)
    print(f"F-Statistic: {f_stat:.2f}")
    print(f"P-Value: {p_value:.4f}")

    if p_value < 0.05:
        print("Final Conclusion: Significant difference found between platforms.")
    else:
        print("Final Conclusion: No significant difference found.")
else:
    print("Assumptions violated. Consider using Kruskal-Wallis H-test instead.")

```

```

Assumptions met. Running One-Way ANOVA...
F-Statistic: 2.56
P-Value: 0.0829
Final Conclusion: No significant difference found.

```

```

groups

```

```

{'TikTok': array([65.70876234, 75.24148111, 53.36094    , 88.31124168, 66.9498208 ,
        62.7274252 , 84.95705692, 73.6630608 , 74.53086367, 80.41976844,
        61.79220387, 74.1663686 , 84.6109146 , 65.48623431, 73.93197654,
        94.78663015, 78.69687923, 63.32408083, 63.86821659, 64.15995633,
        79.18828803, 66.04985321, 62.63865761, 77.99304857, 83.73391146,

```

```

69.32029347, 55.91996341, 51.52365563, 75.99622561, 69.89560614]],
'Instagram': array([76.38405464, 83.48538268, 68.92500799, 77.15189002, 60.37999486,
54.9064827 , 84.64504956, 76.26070475, 51.35595813, 81.21324046,
75.16737898, 66.89979229, 76.945872 , 85.60134782, 58.56430539,
72.17314063, 52.95971012, 73.63052986, 63.54290254, 72.16740738,
79.10188442, 83.40293232, 70.20805162, 58.08966378, 80.21496265,
70.56094365, 70.68869961, 72.30945941, 76.05238316, 71.82903503]),
'LinkedIn': array([74.09639092, 62.94630478, 65.10572241, 81.18655794, 63.9444842 ,
74.01858283, 65.93010531, 78.05168414, 57.61019555, 65.94576657,
73.24615889, 85.56356553, 63.36566193, 65.25667837, 62.22340595,
76.34393328, 77.79381458, 57.40172342, 61.51665284, 63.05031077,
58.25281691, 66.58920115, 73.26048231, 40.97154251, 52.80879973,
67.46343802, 60.08994255, 60.4971181 , 59.24497195, 80.8282937 ]])

```

✓ Problem : Which Training Program Works Best?

Scenario: An HR team at a software company launched 3 different employee training programs (Online, In-Person, Blended). After 1 month, they measured performance scores (out of 100) for 10 employees from each program. Which training style is most effective?

Hypotheses:

H₀ (Null): $\mu_{\text{online}} = \mu_{\text{inperson}} = \mu_{\text{blended}}$ (all three programs yield the same average performance)

H₁ (Alternate): At least one group's mean is different

Assumptions: Normality per group (Shapiro-Wilk) + Homogeneity of Variances (Levene's Test)

```

import numpy as np
from scipy import stats

# Performance scores (out of 100) – 10 employees each
online = np.array([72, 75, 70, 78, 74, 73, 76, 71, 77, 74])
inperson = np.array([85, 88, 82, 90, 86, 87, 84, 89, 83, 88])
blended = np.array([79, 82, 78, 84, 81, 80, 83, 77, 82, 80])

groups = {"Online": online, "In-Person": inperson, "Blended": blended}

```

groups

```

{'Online': array([72, 75, 70, 78, 74, 73, 76, 71, 77, 74]),
 'In-Person': array([85, 88, 82, 90, 86, 87, 84, 89, 83, 88]),
 'Blended': array([79, 82, 78, 84, 81, 80, 83, 77, 82, 80])}

```

Step 1: Assumption 1 – Normality per group

```
print(f"\nAssumption 1 – Normality (Shapiro-Wilk per group)")
```

```

for name, data in groups.items():
    shap_stat, p = shapiro(data)
    print(f"{name}: p_value = {p:.4f}")

```

Step 2: Assumption 2 – Homogeneity of Variance (Levene's)

```
print("\nAssumption 2 – Homogeneity of Variance (Levene's)")
```

```
lev_stat, lev_p = levene(online, inperson, blended)
```

```
print(f"p-value : {lev_p:.4f}  {'✅ Equal variances' if lev_p > 0.05 else '❌ Unequal variances'}")
```

Assumption 1 – Normality (Shapiro-Wilk per group)

Online: p_value = 0.9629

In-Person: p_value = 0.8285

Blended: p_value = 0.9535

Assumption 2 – Homogeneity of Variance (Levene's)

p-value : 0.8007 ✅ Equal variances

```
# Step 4: One-Way ANOVA
```

```
f_stat, p_val = stats.f_oneway(online, inperson, blended)
```

```
print(f"\nOne-Way ANOVA Results")
print(f"F-statistic : {f_stat:.4f}")
print(f"p-value      : {p_val:.6f}")
```

```
One-Way ANOVA Results
F-statistic : 59.9357
p-value      : 0.000000
```

```
print(np.mean(online))
print(np.mean(inperson))
print(np.mean(blended))
```

```
74.0
86.2
80.6
```

A DevOps team wants to compare average file upload speeds (MB/s) across three major cloud providers: AWS S3, Azure Blob Storage, and Google Cloud Storage. They record 20 upload speed measurements from each provider under identical network conditions. The goal is to determine if at least one provider performs differently.

Null Hypothesis (H_0): $\mu_{\text{AWS}} = \mu_{\text{Azure}} = \mu_{\text{GCP}} \rightarrow$ All three cloud providers have the same mean upload speed.

Alternative Hypothesis (H_1): At least one μ differs $\rightarrow$ At least one provider has a significantly different mean upload speed.

```
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt
import pandas as pd
```

```
# Simulated upload speeds (MB/s)
```

```
aws_speed = np.array([ 94.584286 , 94.1104316 , 103.33437439, 84.94608565,
100.96310145, 81.31156991, 93.35308495, 93.12342967,
104.02515235, 94.89899239, 90.0943977 , 105.989508 ,
107.88793581, 89.48617383, 100.53538969, 91.41507518,
96.29873973, 97.05783307, 84.79635313, 95.51203548])
```

```
azure_speed = np.array([ 77.38143382, 78.10631606, 83.42276774, 68.15818386,
73.23557878, 90.3180296 , 94.44159274, 96.52122695,
83.35981281, 94.97176598, 103.67882177, 99.78556206,
74.16043131, 70.52666205, 92.02723791, 100.44482798,
87.76163648, 97.52567708, 90.44963942, 90.2409714 ])
```

```
gcp_speed = np.array([104.07676838, 103.54527371, 99.03689417, 114.91930579,
108.44080198, 98.09458639, 100.00342472, 94.71113355,
105.36258598, 92.00835677, 99.66713507, 107.23115945,
105.9216074 , 106.01596255, 93.73383637, 107.35255058,
101.05930512, 111.27246373, 100.06500648, 113.75685682])
```

```
groups = {'AWS': aws_speed, 'Azure': azure_speed, 'GCP': gcp_speed}
```

```
for name, data in groups.items():
    print(f" {name}  $\rightarrow$  Mean: {data.mean():.2f} MB/s | SD: {data.std():.2f}")
```

```
AWS  $\rightarrow$  Mean: 95.19 MB/s | SD: 6.99
Azure  $\rightarrow$  Mean: 87.33 MB/s | SD: 10.44
GCP  $\rightarrow$  Mean: 103.31 MB/s | SD: 6.18
```

```
groups
```

```
{'AWS': array([ 94.584286 , 94.1104316 , 103.33437439, 84.94608565,
100.96310145, 81.31156991, 93.35308495, 93.12342967,
104.02515235, 94.89899239, 90.0943977 , 105.989508 ,
107.88793581, 89.48617383, 100.53538969, 91.41507518,
96.29873973, 97.05783307, 84.79635313, 95.51203548]),
'Azure': array([ 77.38143382, 78.10631606, 83.42276774, 68.15818386,
73.23557878, 90.3180296 , 94.44159274, 96.52122695,
83.35981281, 94.97176598, 103.67882177, 99.78556206,
74.16043131, 70.52666205, 92.02723791, 100.44482798,
87.76163648, 97.52567708, 90.44963942, 90.2409714 ]),
'GCP': array([104.07676838, 103.54527371, 99.03689417, 114.91930579,
108.44080198, 98.09458639, 100.00342472, 94.71113355,
```

```
105.36258598, 92.00835677, 99.66713507, 107.23115945,
105.9216074 , 106.01596255, 93.73383637, 107.35255058,
101.05930512, 111.27246373, 100.06500648, 113.75685682]]}
```

```
shap_stat_aws, shap_p_val_aws = shapiro(aws_speed)
shap_stat_azure, shap_p_val_azure = shapiro(azure_speed)
shap_stat_gcp, shap_p_val_gcp = shapiro(gcp_speed)
```

```
print(shap_p_val_aws, shap_p_val_azure, shap_p_val_gcp)
```

```
0.8867428130385847 0.3296043672928751 0.8946075226275543
```

```
# — ASSUMPTION 1: Normality —————
print("Assumption 1: Normality (Shapiro-Wilk per Group)")
print()

normality_passed = True

for name, data in groups.items():
    stat, p = shapiro(data)
    print(f"{name} shapiro p-value: {p:.4f}")
    if p < 0.05:
        normality_passed = False
        print(f"--> Warning: {name} data may not be normally distributed.")

print(f"'Normality satisfied for all groups.' if normality_passed else 'Check normality or consider Kruskal-Wallis.'")
```

Assumption 1: Normality (Shapiro-Wilk per Group)

```
AWS shapiro p-value: 0.8867
Azure shapiro p-value: 0.3296
GCP shapiro p-value: 0.8946
Normality satisfied for all groups.
```

```
# — ASSUMPTION 2: Homogeneity of Variance —————
print("Assumption 2: Homogeneity of Variances (Levene's Test)")
print()

lev_stat, lev_p = levene(aws_speed, azure_speed, gcp_speed)

print(f"Levene's → F = {lev_stat:.4f}, p = {lev_p:.4f}")
print(f"'Equal variances - One-Way ANOVA is appropriate.' if lev_p > 0.05 else ' Unequal variances - consider Welch ANOVA.'")
```

Assumption 2: Homogeneity of Variances (Levene's Test)

```
Levene's → F = 3.0093, p = 0.0572
Equal variances - One-Way ANOVA is appropriate.
```

```
# — ONE-WAY ANOVA
print(" One-Way ANOVA")
print()

f_stat, p_val = stats.f_oneway(aws_speed, azure_speed, gcp_speed)
print(f" F-statistic : {f_stat:.4f}")
print(f" p-value      : {p_val:.4f}")

if p_val < 0.05:
    print(" Reject H0 (p < 0.05)")
    print(" At least one provider has a significantly different upload speed.")
else:
    print(" Fail to Reject H0")
```

One-Way ANOVA

```
F-statistic : 18.5959
p-value      : 0.0000
Reject H0 (p < 0.05)
At least one provider has a significantly different upload speed.
```



```

# — VISUALIZATION —————
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle("Cloud Upload Speed: AWS vs Azure vs GCP", fontsize=14, fontweight='bold')
colors = ['#FF9900', '#0078D4', '#4285F4']
# Boxplot
bp = axes[0].boxplot(list(groups.values()), labels=list(groups.keys()),
                    patch_artist=True)
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
axes[0].set_title("Upload Speed Distribution")
axes[0].set_ylabel("Speed (MB/s)")
# Mean plot with error bars
means = [g.mean() for g in groups.values()]
sems = [stats.sem(g) for g in groups.values()]
axes[1].bar(list(groups.keys()), means, yerr=sems, capsize=10,
            color=colors, alpha=0.8, edgecolor='black')
axes[1].set_title("Mean Upload Speed (±SEM)")
axes[1].set_ylabel("Speed (MB/s)")
plt.tight_layout()
plt.savefig("anova_problem1.png", dpi=120, bbox_inches='tight')
plt.show()
print("\n Plot saved as anova_problem1.png")

```

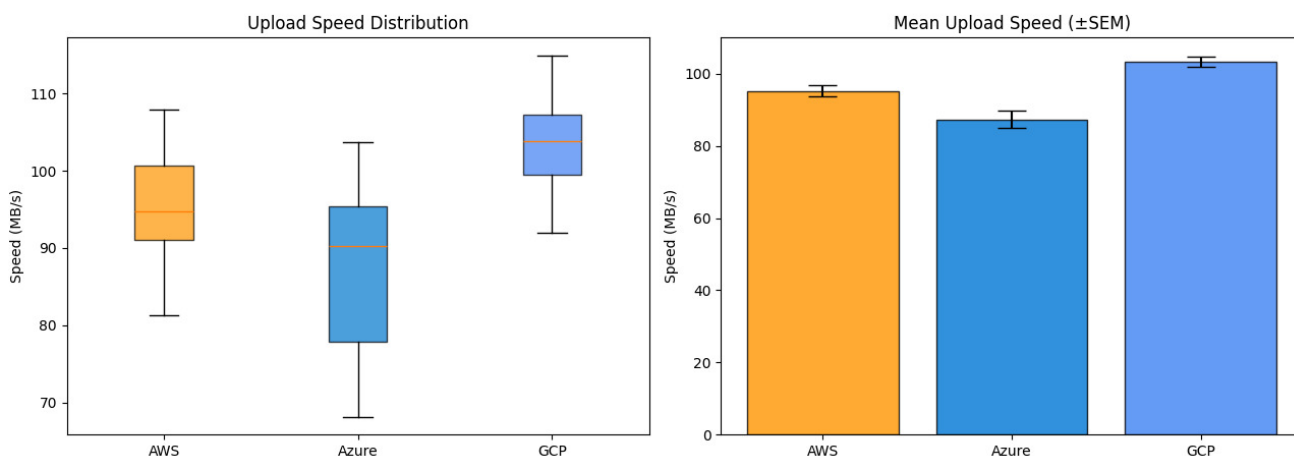
/tmp/ipykernel_480/586216033.py:6: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels', with the same default value. This will become a hard error in the next major release.

```

bp = axes[0].boxplot(list(groups.values()), labels=list(groups.keys()),

```

Cloud Upload Speed: AWS vs Azure vs GCP



Plot saved as anova_problem1.png

Note: If the assumptions of ANOVA test are not met then you use **Kruskal Test**

How to Interpret the results

1. If all Shapiro-Wilk p-values > 0.05: Groups are normally distributed.
2. If Levene's p-value > 0.05: Variances are equal (homogeneity holds).
3. If ANOVA p-value < 0.05: At least one group mean is significantly different.

✓ Chi-Squared Test for independence for categorical samples

Problem 1 : Does Ad Channel Affect Purchase Decision?

Scenario: A D2C (Direct-to-Consumer) brand ran ads across 3 channels — Instagram, YouTube, and Google. They want to know if the channel a customer saw the ad on has any relationship with whether they made a purchase or not. If there's a significant association, the marketing team will double down on the best-performing channel.

Variables:

Variable 1 (Categorical): Ad Channel → Instagram, YouTube, Google

Variable 2 (Categorical): Purchase Decision → Purchased, Not Purchased

Hypotheses:

H₀ (Null): Ad channel and purchase decision are independent (the channel doesn't affect buying behavior)

H₁ (Alternate): Ad channel and purchase decision are NOT independent (the channel significantly influences purchases)

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# -----
# Step 1: Build the contingency table
# Rows → Ad Channel : Instagram, YouTube, Google
# Columns → Decision : Purchased, Not Purchased
# -----
observed = np.array([
    [80, 120], # Instagram
    [95, 55], # YouTube
    [60, 90], # Google
])

observed

array([[ 80, 120],
       [ 95,  55],
       [ 60,  90]])
```

```
df_table = pd.DataFrame(
    observed,
    index=["Instagram", "YouTube", "Google"],
    columns=["Purchased", "Not Purchased"]
)
df_table["Row Total"] = df_table.sum(axis=1)
df_table.loc["Col Total"] = df_table.sum()
```

df_table

	Purchased	Not Purchased	Row Total
Instagram	80	120	200
YouTube	95	55	150
Google	60	90	150
Col Total	235	265	500

```
chi2, p_val, dof, expected = chi2_contingency(observed)

print('P-value for Chi-Squared test: ', p_val)

P-value for Chi-Squared test: 1.039025672094255e-05
```

Problem Statement 2: A restaurant chain wants to determine if there is an association between the type of cuisine offered (Italian, Chinese, Mexican) and customer satisfaction levels (satisfied, neutral, dissatisfied). They conduct a survey among customers across several locations to collect this data.

Hypotheses:

- Null Hypothesis (H_0): There is no association between the type of cuisine and customer satisfaction levels. The variables are independent.
- Alternative Hypothesis (H_1): There is an association between the type of cuisine and customer satisfaction levels. The variables are not independent.

```

import numpy as np
from scipy.stats import chi2_contingency

# Observed frequency data in a contingency table
# Rows: Type of Cuisine (Italian, Chinese, Mexican)
# Columns: Customer Satisfaction (Satisfied, Neutral, Dissatisfied)
data = np.array([[40, 30, 10], # Italian
                 [35, 25, 20], # Chinese
                 [25, 30, 15]]) # Mexican

# Perform Chi-Square Test of Independence
chi2_stat, p_value, dof, expected = chi2_contingency(data)

# Interpret the results
alpha = 0.05
if p_value < alpha:
    print(f"Chi-Square Statistic: {chi2_stat:.4f}, p-value: {p_value:.4f}")
    print("We have sufficient evidence to reject the null hypothesis.")
    print("There is a significant association between the type of cuisine and customer satisfaction levels.")
else:
    print(f"Chi-Square Statistic: {chi2_stat:.4f}, p-value: {p_value:.4f}")
    print("We do not have sufficient evidence to reject the null hypothesis.")
    print("There is no significant association between the type of cuisine and customer satisfaction levels.")

```

Scenario 3: Banking Risk Management (Credit Default)

Problem Statement: A commercial bank is analyzing its loan portfolio. The Risk Management team wants to understand if a customer's Housing Status (Own, Mortgage, Rent) influences their Loan Default Status (Defaulted, Paid Off). If there is a strong dependency, housing status will be weighted more heavily in the bank's future credit scoring models.

Hypothesis Statements: